PWA Tutorial

A Complete Guide

25 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

https://snehaliteng.github.io/

Copyright & Disclaimer

> **Please do not print this document unnecessarily.** Think of the environment — save paper and trees. Share the digital link instead: https://snehaliteng.github.io/tutorials/pwa/

CHAPTER 1

Welcome to Progressive Web Apps

What are Progressive Web Apps?

Progressive Web Apps (PWAs) are web applications that use modern web capabilities to deliver an app-like experience to users. They are reliable, fast, and engaging, combining the best of web and native apps.

> **Core Principle:** PWAs are websites that progressively evolve to feel like native apps. They work offline, load instantly, and can be installed on the user's device.

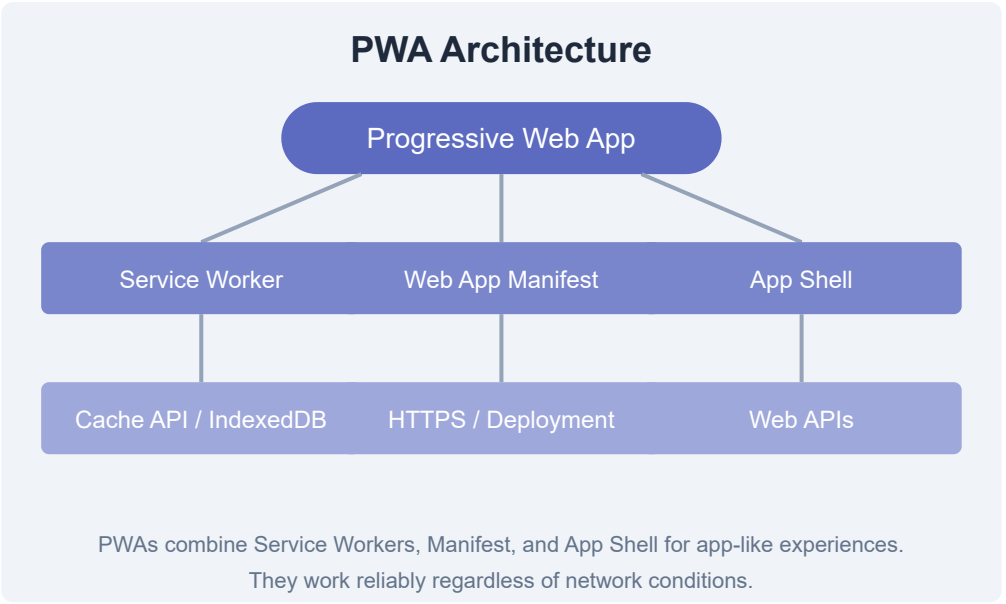
Why PWAs?

- **Reliable:** Load instantly, even on slow networks or offline.
- **Fast:** Respond quickly to user interactions.
- **Engaging:** Feel like a natural app with an immersive experience.
- **No Install Friction:** Users can try the PWA without installing from an app store.
- **Cross-Platform:** One codebase for all platforms and devices.

How PWAs Work

PWAs are built on three core technologies:

1. **Service Worker:** A JavaScript file that runs in the background, enabling offline functionality, caching, and push notifications.
2. **Web App Manifest:** A JSON file that controls how the PWA appears when installed, including name, icons, and display mode.
3. **App Shell:** The minimal HTML, CSS, and JavaScript needed to render the user interface, cached for instant loading.



PWAs combine Service Workers, Manifest, and App Shell for app-like experiences. They work reliably regardless of network conditions.

Progressive Enhancement Philosophy

PWAs embrace progressive enhancement — they work for every user regardless of browser choice, but deliver an enhanced experience for users on modern browsers. This means your PWA is still a functional website for everyone.

Browser Support

PWAs are supported in Chrome, Firefox, Edge, Safari (starting with iOS 16.4+), and Samsung Internet. Key features like Service Workers and Web App Manifest are widely supported across modern browsers.

> **Exercise:** Visit a well-known PWA like Twitter, Pinterest, or Spotify on your mobile browser. Notice the install prompt, offline behavior, and app-like navigation. Write down three observations about the user experience.

CHAPTER 2

Getting Started with PWAs

Prerequisites

- Basic knowledge of HTML, CSS, and JavaScript
- A code editor (VS Code recommended)
- A web server (local or remote) — many PWA features require HTTPS
- Modern browser (Chrome, Edge, or Firefox)

Development Environment

```
# Option 1: Use VS Code + Live Server
npm install -g live-server
live-server --https

# Option 2: Use Node.js http-server with HTTPS
npm install -g http-server
http-server -S -C cert.pem -o

# Option 3: Use Python
python -m http.server 8000

# For local HTTPS (recommended for PWA dev):
# Install mkcert
mkcert -install
mkcert localhost
# Then use the generated cert.pem and key.pem
```

Your First PWA

Let's create a minimal PWA step by step.

1. Create the HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First PWA</title>
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#4f46e5">
  <link rel="apple-touch-icon" href="icon-192.png">
</head>
<body>
  <h1>Hello, PWA!</h1>
  <p>This is my first Progressive Web App.</p>
  <script src="app.js"></script>
</body>
</html>
```

2. Create the Web App Manifest

```
{
  "name": "My First PWA",
  "short_name": "PWA Demo",
  "description": "A simple demo PWA",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#4f46e5",
  "icons": [
    {
      "src": "icon-192.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icon-512.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
```

3. Register a Service Worker

```
// app.js
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('/sw.js')
      .then(reg => console.log('SW registered:', reg.scope))
      .catch(err => console.error('SW registration failed:', err));
  });
}
```

4. Create the Service Worker

```
// sw.js
self.addEventListener('install', (event) => {
  console.log('Service Worker installing...');
  self.skipWaiting();
});

self.addEventListener('activate', (event) => {
  console.log('Service Worker activating...');
});

self.addEventListener('fetch', (event) => {
  console.log('Fetching:', event.request.url);
});
```

Important: Service Workers only work over HTTPS (or localhost for development). You must serve your PWA over a secure connection.

Exercise: Create the four files above (index.html, manifest.json, app.js, sw.js) on your local machine. Serve them with a local HTTPS server, open Chrome DevTools, and verify the Service Worker is installed in Application > Service Workers.

CHAPTER 3

PWA Foundations

Core Concepts

Understanding these foundational concepts is essential for building effective PWAs:

The App Shell Model

App Shell architecture separates the core application shell (UI infrastructure) from the dynamic content. The shell is cached on first load and instantly renders on subsequent visits.

```
<!-- app-shell.html -->
<!DOCTYPE html>
<html>
<head>
  <!-- Shell resources -->
  <link rel="stylesheet" href="/css/app-shell.css">
  <link rel="stylesheet" href="/css/content.css">
</head>
<body>
  <header>
    <nav id="main-nav">...</nav>
  </header>

  <main id="content">
    <!-- Dynamic content loads here -->
  </main>

  <footer>...</footer>

  <script src="/js/app-shell.js"></script>
  <script src="/js/content.js"></script>
</body>
</html>
```

HTTPS Requirement

Service Workers require secure contexts. This means your PWA must be served over HTTPS. HTTPS ensures the integrity of the Service Worker and prevents man-in-the-middle attacks.

> **Exception:** Localhost is treated as a secure context for development purposes.

PWA Ecosystem

- **Service Workers:** Background scripts that enable offline functionality.
- **Cache API:** A storage system for Request/Response pairs.
- **IndexedDB:** Client-side NoSQL database for structured data.
- **Web App Manifest:** JSON metadata for installable web apps.

- **Push API:** Server-to-client push notifications.
- **Background Sync:** Defer actions until connectivity is available.

Best Practices

- Always use HTTPS
- Implement responsive design
- Provide a 404 offline page
- Use semantic HTML and ARIA for accessibility
- Optimize images and assets
- Test on real devices
- Ensure backward compatibility
- Use a privacy-friendly approach

> **Exercise:** Audit your current website (or a simple HTML page) using Lighthouse in Chrome DevTools. Check the PWA section and identify which criteria it passes and fails. Document your findings.

CHAPTER 4

App Design for PWAs

UI/UX Design Principles

PWAs should feel like natural apps. Design principles include:

- **Mobile-first:** Design for small screens first, then enhance for larger screens.
- **Fast feedback:** Respond to user actions within 100ms.
- **Smooth transitions:** Use CSS animations and transitions for fluid navigation.
- **Offline-aware:** Gracefully handle offline states with informative UI.

Responsive Layouts

Use CSS Grid, Flexbox, and media queries to create adaptive layouts that work on all screen sizes.

```
/* Responsive app layout */
.app-layout {
  display: grid;
  grid-template-areas:
    "header"
    "main"
    "footer";
  grid-template-rows: 56px 1fr auto;
  min-height: 100vh;
}

@media (min-width: 768px) {
  .app-layout {
    grid-template-columns: 240px 1fr;
    grid-template-areas:
      "header header"
      "sidebar main"
      "sidebar footer";
  }
}

/* Touch-friendly targets */
button, a, input {
  min-height: 48px;
  min-width: 48px;
}

/* Pull-to-refresh simulation */
.pull-to-refresh {
  touch-action: pan-y;
  overscroll-behavior: contain;
}
```

Accessibility (A11y)

PWAs must be accessible to all users:

- Use semantic HTML elements (`<nav>`, `<main>`, `<article>`)
- Provide ARIA labels for interactive elements
- Ensure color contrast ratios (WCAG AA: 4.5:1)
- Support keyboard navigation

- Test with screen readers (VoiceOver, NVDA)

```
<!-- Accessible navigation -->
<nav aria-label="Main navigation" role="navigation">
  <button aria-label="Open menu">
    <span class="hamburger" aria-hidden="true"></span>
  </button>
  <ul role="menubar">
    <li role="none">
      <a role="menuitem" href="/" aria-current="page">Home</a>
    </li>
  </ul>
</nav>
```

App-like Look & Feel

- Use system fonts (`-apple-system`, `Segoe UI`) for native feel
- Add safe-area-inset for notched devices
- Use CSS `env(safe-area-inset-*)` for proper padding
- Implement swipe gestures (with libraries like Swiper or Hammer.js)

```
/* Safe area support for notched devices */
body {
  padding-top: env(safe-area-inset-top);
  padding-bottom: env(safe-area-inset-bottom);
  padding-left: env(safe-area-inset-left);
  padding-right: env(safe-area-inset-right);
}

/* System font stack */
body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI',
    Roboto, Oxygen, Ubuntu, Cantarell, sans-serif;
}
```

Exercise: Design a responsive PWA layout for a weather app. Create the HTML and CSS with a header, main content area with a 7-day forecast grid, and a bottom navigation bar. Make it touch-friendly and accessible.

CHAPTER 5

Assets & Data Management

Static Assets

Static assets include HTML, CSS, JavaScript, images, fonts, and other files that don't change frequently. These are prime candidates for caching.

```
// sw.js - Cache static assets on install
const CACHE_NAME = 'v1';
const STATIC_ASSETS = [
  '/',
  '/index.html',
  '/css/styles.css',
  '/js/app.js',
  '/images/logo.svg',
  '/fonts/inter.woff2',
  '/offline.html'
];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => cache.addAll(STATIC_ASSETS))
      .then(() => self.skipWaiting())
  );
});
```

Data Management

Client-side Storage Options

Storage	Use Case	Limit
localStorage	Simple key-value, small data	~5MB
Cache API	Network requests, responses	Disk quota
IndexedDB	Structured data, large datasets	Disk quota

Optimization

- **Image optimization:** Use WebP/AVIF, lazy loading, responsive images
- **Font optimization:** Subset fonts, use woff2, font-display: swap
- **Code splitting:** Load only what's needed
- **Tree shaking:** Remove unused code
- **Minification:** Minify HTML, CSS, and JS

```
<!-- Responsive images with WebP fallback -->
<picture>
  <source srcset="photo.avif" type="image/avif">
  <source srcset="photo.webp" type="image/webp">
  <img src="photo.jpg" alt="Description" loading="lazy">
</picture>

<!-- Font display optimization -->
<style>
  @font-face {
    font-family: 'CustomFont';
    src: url('/fonts/custom.woff2') format('woff2');
    font-display: swap; /* Show fallback font immediately */
  }
</style>
```

Exercise: Audit the assets of a simple web page using Chrome DevTools' Coverage tab. Identify unused CSS and JavaScript, then create a Service Worker that precaches only the critical assets needed for the first meaningful paint.

CHAPTER 6

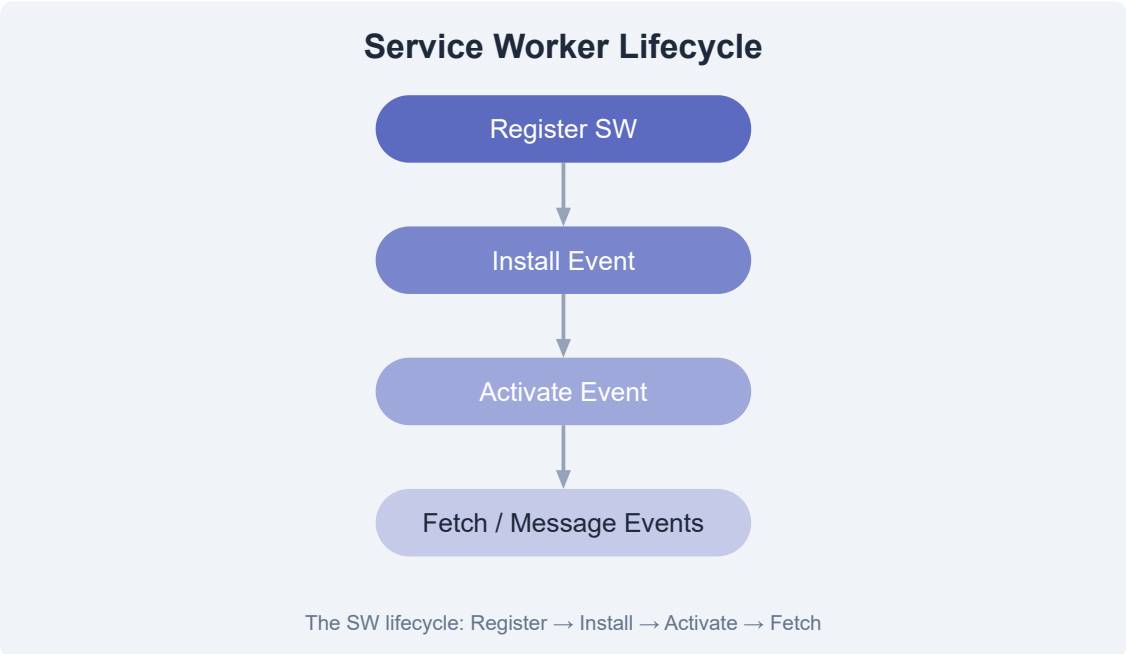
Service Workers

What is a Service Worker?

A Service Worker is a JavaScript file that runs in the background, separate from the web page. It acts as a programmable network proxy, intercepting network requests and enabling offline functionality.

> **Key Characteristics:** Runs on its own thread, has no DOM access, works offline, and is only active when needed (event-driven).

Service Worker Lifecycle



The SW lifecycle: Register → Install → Activate → Fetch

1. **Registration:** The page registers the Service Worker file.
2. **Install:** Fired when the browser installs the SW. Ideal for precaching assets.
3. **Activate:** Fired after installation. Used for cleanup and claiming clients.

4. **Fetch/Message:** The SW intercepts network requests and listens for messages.

Registration

```
// Register the Service Worker
if ('serviceWorker' in navigator) {
  window.addEventListener('load', async () => {
    try {
      const registration = await navigator.serviceWorker.register('/sw.js', {
        scope: '/'  // Controls which pages the SW controls
      });
      console.log('SW registered:', registration.scope);

      // Check for updates
      registration.addEventListener('updatefound', () => {
        const newWorker = registration.installing;
        console.log('New SW found:', newWorker);
      });
    } catch (err) {
      console.error('SW registration failed:', err);
    }
  });
}
```

Event Handling

```
// sw.js - Full lifecycle example
const CACHE = 'my-pwa-v1';

// Install event
self.addEventListener('install', (event) => {
  console.log('SW: Installing...');
  event.waitUntil(
    caches.open(CACHE).then(cache => {
      return cache.addAll([
        '/',
        '/index.html',
        '/css/styles.css',
        '/js/app.js',
        '/offline.html'
      ]);
    })
  );
  self.skipWaiting();
});

// Activate event
self.addEventListener('activate', (event) => {
  console.log('SW: Activating...');
  event.waitUntil(
    caches.keys().then(keys => {
      return Promise.all(
        keys.filter(key => key !== CACHE)
          .map(key => caches.delete(key))
      );
    })
  );
  self.clients.claim();
});

// Fetch event - intercept network requests
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request)
      .then(cached => {
        // Return cached response or fetch from network
        const fetchPromise = fetch(event.request).then(response => {
```

```
          // Cache the new response
          return caches.open(CACHE).then(cache => {
            cache.put(event.request, response.clone());
            return response;
          });
        });
        return cached || fetchPromise;
      })
  );
});

// Message event - listen for messages from the page
self.addEventListener('message', (event) => {
  if (event.data.type === 'SKIP_WAITING') {
    self.skipWaiting();
  }
});
```

Scope and Control

The `scope` option in registration defines which pages the Service Worker controls. By default, the scope is the directory containing the SW file. To expand scope, set the `Service-Worker-Allowed` HTTP header.

> **Exercise:** Create a Service Worker that logs all fetch events. Register it on a simple HTML page, open DevTools > Application > Service Workers, and observe the lifecycle events as you reload the page.

CHAPTER 7

Caching Strategies

Cache API

The Cache API provides a storage mechanism for Request and Response objects. It's the foundation for all offline strategies.

```
// Open or create a cache
caches.open('my-cache-v1').then(cache => {
  // Add a single request/response
  cache.add('/api/data.json');

  // Add multiple URLs
  cache.addAll(['/css/styles.css', '/js/app.js']);

  // Manually put a response
  cache.put('/api/data.json', new Response('{"status":"ok"}'));

  // Match a request
  cache.match('/api/data.json').then(response => {
    if (response) { /* use cached response */ }
  });

  // Delete from cache
  cache.delete('/old-asset.js');
});

// List all caches
caches.keys().then(names => console.log(names));

// Delete an entire cache
caches.delete('old-cache-v1');
```

Common Caching Strategies

1. Cache First (Offline-first)

Return cached content if available; otherwise fetch from network. Best for static assets.

```
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request)
      .then(cached => cached || fetch(event.request))
  );
});
```

2. Network First

Try the network first; fall back to cache if offline. Best for frequently updated content.

```
self.addEventListener('fetch', (event) => {
  event.respondWith(
    fetch(event.request)
      .then(response => {
        return caches.open('dynamic').then(cache => {
          cache.put(event.request, response.clone());
          return response;
        });
      })
      .catch(() => caches.match(event.request))
  );
});
```

3. Stale-While-Revalidate

Return cached content immediately, then update the cache with the network response. Best for content that doesn't need instant freshness.

```
self.addEventListener('fetch', (event) => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      const fetchPromise = fetch(event.request).then(response => {
        return caches.open('dynamic').then(cache => {
          cache.put(event.request, response.clone());
          return response;
        });
      });
      return cached || fetchPromise;
    })
  );
});
```

4. Network Only / Cache Only

```
// Network only - always fetch from network
event.respondWith(fetch(event.request));

// Cache only - always serve from cache (fail if not cached)
event.respondWith(caches.match(event.request));
```

Advanced Patterns

```
// Pattern: Cache with network update
self.addEventListener('fetch', (event) => {
  if (event.request.url.includes('/api/')) {
    event.respondWith(
      caches.open('api-cache').then(cache => {
        return cache.match(event.request).then(cached => {
          const networkPromise = fetch(event.request).then(response => {
            cache.put(event.request, response.clone());
            return response;
          });
          return cached || networkPromise;
        });
      })
    );
  }
});
```

Exercise: Implement three different caching strategies for a single-page app: cache-first for static assets (CSS/JS), network-first for blog posts API, and stale-while-revalidate for avatar images. Verify behavior in DevTools under offline mode.

CHAPTER 8

Serving Content

Hosting Your PWA

Your PWA can be hosted on any static file server. Popular options include:

- **GitHub Pages:** Free for open-source projects
- **Netlify:** Free tier with HTTPS, deploy from Git
- **Vercel:** Optimized for frontend frameworks
- **Firebase Hosting:** Google's CDN-backed hosting
- **Azure Static Web Apps:** Integrated with Azure services
- **Any HTTPS server:** Nginx, Apache, Caddy

HTTPS Configuration

Service Workers require HTTPS. Here's how to set it up on common platforms:

```
# Nginx HTTPS configuration
server {
    listen 443 ssl;
    server_name my-pwa.example.com;

    ssl_certificate /etc/letsencrypt/live/my-pwa/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/my-pwa/privkey.pem;

    root /var/www/my-pwa;
    index index.html;

    # Important: Serve Service Worker with correct headers
    location /sw.js {
        add_header Service-Worker-Allowed /;
        add_header Cache-Control "no-cache";
        add_header Content-Type "application/javascript";
    }

    # Cache static assets
    location /static/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

CDN and Delivery

Content Delivery Networks (CDNs) can significantly improve load times for users around the world. When using a CDN with a PWA:

- Ensure the Service Worker is served from the same origin as the page
- Use `Cache-Control` headers properly
- Cross-origin resources should not be cached by SW unless CORS is properly configured

Security Considerations

- **Content Security Policy (CSP):** Add CSP headers to prevent XSS

- **Subresource Integrity (SRI):** Ensure CDN assets haven't been tampered with
- **HTTPS:** Redirect HTTP to HTTPS
- **HSTS:** Use Strict-Transport-Security header

```
# Security headers for PWA
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; st
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; pr
add_header X-Content-Type-Options "nosniff";
add_header X-Frame-Options "DENY";
```

> **Exercise:** Deploy your PWA to a free hosting service (Netlify, Vercel, or GitHub Pages). Verify that HTTPS is enabled, the Service Worker registers successfully, and the site works offline after the first visit.

CHAPTER 9

Workbox

What is Workbox?

Workbox is a library created by Google that simplifies Service Worker development. It provides production-ready caching strategies, routing, precaching, and background sync — all with minimal boilerplate.

Setup

```
# Install Workbox CLI
npm install workbox-cli --global

# Or add to your project
npm install workbox-webpack-plugin --save-dev
npm install workbox-sw --save
```

Using Workbox via CDN

```
// sw.js - Using Workbox from CDN
importScripts('https://storage.googleapis.com/workbox-cdn/releases/7.0.0/work

if (workbox) {
  console.log('Workbox loaded successfully');
} else {
  console.log('Workbox failed to load');
}
```

Precaching with Workbox

```
// sw.js with workbox-build (recommended approach)
// Use workbox-webpack-plugin or workbox-build to generate this

const { precacheAndRoute } = workbox.precaching;
precacheAndRoute(self.__WB_MANIFEST);

// The manifest is auto-generated by workbox-build
// It lists all static assets to precache
```

Runtime Caching Strategies

```
// Register routes with different strategies
workbox.routing.registerRoute(
  /\.(?:js|css|html)$/,
  new workbox.strategies.StaleWhileRevalidate({
    cacheName: 'static-resources'
  })
);

workbox.routing.registerRoute(
  /\.(?:png|jpg|jpeg|svg|gif|webp)$/,
  new workbox.strategies.CacheFirst({
    cacheName: 'image-cache',
    plugins: [
      new workbox.expiration.ExpirationPlugin({
        maxEntries: 60,
        maxAgeSeconds: 30 * 24 * 60 * 60, // 30 days
      }),
      new workbox.cacheableResponse.CacheableResponsePlugin({
        statuses: [0, 200]
      })
    ]
  })
);

workbox.routing.registerRoute(
  /\/api\//,
  new workbox.strategies.NetworkFirst({
    cacheName: 'api-cache',
    plugins: [
      new workbox.expiration.ExpirationPlugin({
        maxEntries: 50,
        maxAgeSeconds: 5 * 60, // 5 minutes
      })
    ]
  })
);
```

Workbox Webpack Plugin

```
// webpack.config.js
const { InjectManifest } = require('workbox-webpack-plugin');

module.exports = {
  plugins: [
    new InjectManifest({
      swSrc: './src/sw.js',
      swDest: 'sw.js',
      maximumFileSizeToCacheInBytes: 5 * 1024 * 1024,
    })
  ]
};
```

Advanced Features

```
// Background Sync
const bgSyncPlugin = new workbox.backgroundSync.BackgroundSyncPlugin('my-queu
  maxRetentionTime: 24 * 60 // Retry for 24 hours
});

workbox.routing.registerRoute(
  /\/api\/sync\//,
  new workbox.strategies.NetworkOnly({
    plugins: [bgSyncPlugin]
  }),
  'POST'
);

// Broadcast Update (notify page when cached content updates)
workbox.routing.registerRoute(
  /\.(?:js|css)$/,
  new workbox.strategies.StaleWhileRevalidate({
    cacheName: 'static-assets',
    plugins: [
      new workbox.broadcastUpdate.BroadcastUpdatePlugin({
        channelName: 'precache-updates'
      })
    ]
  })
);
```

Exercise: Set up Workbox in a project using the Webpack plugin. Configure precaching for all static assets, a CacheFirst strategy for images with a 30-day expiration, and a NetworkFirst strategy for API calls. Build and verify the generated Service Worker.

CHAPTER 10

Offline Data Management

Storage Options

PWAs have several options for storing data on the client:

API	Type	Best For
localStorage	Sync key-value	Preferences, small data
sessionStorage	Sync key-value	Session-only data
IndexedDB	Async NoSQL	Complex data, large datasets
Cache API	HTTP responses	Network response caching

IndexedDB

IndexedDB is a low-level, asynchronous NoSQL database in the browser. It can store large amounts of structured data, including files and blobs.

```
// Open a database
const request = indexedDB.open('MyPWA', 1);

request.onupgradeneeded = (event) => {
  const db = event.target.result;
  // Create object store
  const store = db.createObjectStore('tasks', {
    keyPath: 'id',
    autoIncrement: true
  });
  // Create indexes
  store.createIndex('status', 'completed', { unique: false });
  store.createIndex('dueDate', 'dueDate', { unique: false });
};

request.onsuccess = (event) => {
  const db = event.target.result;
  console.log('Database opened successfully');
};

// CRUD operations
function addTask(task) {
  const db = indexedDB.open('MyPWA', 1);
  db.onsuccess = (event) => {
    const db = event.target.result;
    const tx = db.transaction('tasks', 'readwrite');
    const store = tx.objectStore('tasks');
    store.add(task);
    tx.oncomplete = () => console.log('Task added');
  };
}

function getTasks() {
  return new Promise((resolve, reject) => {
    const db = indexedDB.open('MyPWA', 1);
    db.onsuccess = (event) => {
      const db = event.target.result;
      const tx = db.transaction('tasks', 'readonly');
      const store = tx.objectStore('tasks');
      const all = store.getAll();
      all.onsuccess = () => resolve(all.result);
      all.onerror = () => reject(all.error);
    };
```

```
  });
}
```

Using IndexedDB with Dexie.js (Simplified)

```
// Install: npm install dexie

import Dexie from 'dexie';

const db = new Dexie('MyPWA');

// Define schema
db.version(1).stores({
  tasks: '++id, name, completed, dueDate',
  notes: '++id, title, &url'
});

// CRUD
await db.tasks.add({ name: 'Learn PWAs', completed: false });
const tasks = await db.tasks.where('completed').equals(false).toArray();
await db.tasks.update(1, { completed: true });
await db.tasks.delete(1);
```

Background Sync

The Background Sync API allows you to defer actions until the user has stable connectivity.

```
// Register a sync event
async function registerSync() {
  const registration = await navigator.serviceWorker.ready;
  await registration.sync.register('sync-tasks');
}

// In Service Worker
self.addEventListener('sync', (event) => {
  if (event.tag === 'sync-tasks') {
    event.waitUntil(syncPendingTasks());
  }
});

async function syncPendingTasks() {
  const tasks = await getOfflineTasks();
  for (const task of tasks) {
    await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(task)
    });
  }
}
```

> **Exercise:** Build an offline-capable to-do list app that stores tasks in IndexedDB. When the user creates a task while offline, save it locally and sync it to a server when connectivity returns using Background Sync.

CHAPTER 11

Installation

Web App Manifest

The Web App Manifest is a JSON file that tells the browser about your PWA and how it should behave when installed.

```
{
  "name": "My Progressive Web App",
  "short_name": "MyPWA",
  "description": "A description of my PWA",
  "start_url": "/dashboard",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#4f46e5",
  "orientation": "portrait-primary",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any maskable"
    },
    {
      "src": "/icons/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any maskable"
    }
  ],
  "categories": ["productivity", "utilities"],
  "lang": "en-US",
  "scope": "/",
  "id": "/?source=pwa"
}
```

Install Criteria

Browsers have criteria that must be met before they prompt the user to install the PWA:

- Web App Manifest with `short_name`, `name`, and `icons` (including a 192px and 512px icon)
- `start_url` defined
- `display` set to `standalone` or `minimal-ui`
- Served over HTTPS
- Registered Service Worker with a fetch event handler
- User has visited the app for at least 5 minutes (Chrome)

Desktop PWA Support

PWAs can also be installed on desktop (Windows, macOS, Linux, ChromeOS). The experience includes:

- Standalone window with title bar
- Start menu / Dock integration
- Taskbar / Dock shortcuts
- Protocol handling
- File handling

Installation Customization

```
// Listen for the install prompt
let deferredPrompt = null;

window.addEventListener('beforeinstallprompt', (event) => {
  // Prevent the mini-infobar from appearing
  event.preventDefault();
  // Store the event so it can be triggered later
  deferredPrompt = event;
  // Show the install button
  installButton.style.display = 'block';
});

// Custom install button
installButton.addEventListener('click', async () => {
  if (deferredPrompt) {
    deferredPrompt.prompt();
    const result = await deferredPrompt.userChoice;
    console.log('User response:', result.outcome);
    deferredPrompt = null;
    installButton.style.display = 'none';
  }
});
```

Exercise: Create a manifest.json for a "Note Taking" PWA. Include icons of different sizes (192x192, 512x512), set the display to standalone, define a theme color, and add the categories property. Test it in Chrome DevTools > Application > Manifest.

CHAPTER 12

Manifest Deep Dive

All Manifest Properties

Property	Description	Required
name	Human-readable app name	Yes
short_name	Short name for the home screen	Recommended
description	Description of the app	Optional
start_url	URL loaded when launched	Yes
display	Display mode (fullscreen, standalone, minimal-ui, browser)	Yes
orientation	Default orientation	Optional
icons	Array of image objects	Yes
background_color	Splash screen background	Recommended
theme_color	Toolbar/taskbar color	Recommended
scope	Navigation scope of the app	Optional
id	Unique identifier for the app	Optional
categories	App store categories	Optional
iarc_rating_id	Content rating ID	Optional
screenshots	App screenshots for store listing	Optional

Icons and Maskable Icons

Maskable icons are adaptive icons that fill the entire shape of the icon mask on Android. They ensure your icon looks great across different devices.

```
// SVG as maskable icon (concept)
// The safe zone is the inner 80% of the icon
// The outer 20% may be cropped by the mask

{
  "src": "/icons/icon-512.png",
  "sizes": "512x512",
  "type": "image/png",
  "purpose": "any maskable"
}
```

Splash Screen

The splash screen is generated automatically from the manifest properties:

- Uses `background_color` as the background
- Uses the largest icon (ideally 512x512) centered
- Shows `name` or `short_name`

Display Modes

Mode	Description
fullscreen	Hides all browser UI, fills the screen
standalone	Looks like a native app, hides browser chrome
minimal-ui	Shows minimal browser controls (back, forward, refresh)
browser	Opens in the regular browser tab

Manifest Linking

```
<!-- In the HTML head -->
<link rel="manifest" href="/manifest.json">

<!-- Apple-specific meta tags (for iOS) -->
<meta name="apple-mobile-web-app-capable" content="yes">
<meta name="apple-mobile-web-app-status-bar-style" content="black-translucent
<meta name="apple-mobile-web-app-title" content="My PWA">
<link rel="apple-touch-icon" href="/icons/icon-192.png">
<link rel="apple-touch-startup-image" href="/splash.png">

<!-- Windows / IE meta tags -->
<meta name="application-name" content="My PWA">
<meta name="msapplication-TileColor" content="#4f46e5">
<meta name="msapplication-TileImage" content="/icons/icon-144.png">
```

> **Exercise:** Audit an existing PWA's manifest using Chrome DevTools. Check all properties, verify icon sizes and purposes, and test how the splash screen looks. Modify the manifest to add maskable icons and screenshots.

CHAPTER 13

Install Prompt

beforeinstallprompt Event

The `beforeinstallprompt` event fires when the browser determines the PWA is installable. It allows you to intercept the default install prompt and show a custom one.

```
let deferredPrompt = null;

window.addEventListener('beforeinstallprompt', (event) => {
  // Prevent the mini-infobar from appearing
  event.preventDefault();
  // Store for later use
  deferredPrompt = event;
  // Show a custom install button
  showInstallButton();
});

// Handle the install button click
async function handleInstallClick() {
  if (!deferredPrompt) return;

  // Show the browser's install prompt
  deferredPrompt.prompt();

  // Wait for user response
  const { outcome } = await deferredPrompt.userChoice;

  if (outcome === 'accepted') {
    console.log('User installed the PWA');
    trackInstallEvent('accepted');
  } else {
    console.log('User dismissed the install prompt');
    trackInstallEvent('dismissed');
  }

  // Clean up
  deferredPrompt = null;
  hideInstallButton();
}
```

Custom Install Prompts

Create a branded install prompt that matches your app's design:

```
<!-- Custom install prompt HTML -->
<div id="install-prompt" class="install-banner" hidden>
  <div class="install-content">
    <img src="/icons/icon-192.png" alt="App icon">
    <div>
      <h3>Install My PWA</h3>
      <p>Add to your home screen for the best experience</p>
    </div>
    <button id="install-btn">Install</button>
    <button id="dismiss-btn">Not now</button>
  </div>
</div>
```

```
/* Custom install prompt styles */
.install-banner {
  position: fixed;
  bottom: 0;
  left: 0;
  right: 0;
  background: white;
  padding: 16px;
  box-shadow: 0 -2px 10px rgba(0,0,0,0.1);
  z-index: 1000;
  animation: slideUp 0.3s ease;
}

@keyframes slideUp {
  from { transform: translateY(100%); }
  to { transform: translateY(0); }
}

.install-content {
  display: flex;
  align-items: center;
  gap: 12px;
}

.install-content img {
  width: 48px;
  height: 48px;
  border-radius: 12px;
}

#install-btn {
  background: #4f46e5;
  color: white;
  border: none;
  padding: 8px 20px;
  border-radius: 20px;
  font-weight: 600;
  cursor: pointer;
  white-space: nowrap;
}
```

Deferred Prompts

Best practices for deferred install prompts:

- Don't show the prompt immediately on page load
- Show it after the user has completed a meaningful action
- Don't show it if the user has already installed or dismissed it
- Respect the user's choice — don't repeatedly prompt
- Provide a clear benefit for installing

Analytics and Tracking

```
function trackInstallEvent(outcome) {
  // Send to analytics
  if (typeof gtag !== 'undefined') {
    gtag('event', 'install_prompt', {
      event_category: 'PWA',
      event_label: outcome,
      value: 1
    });
  }

  // Store dismissal in localStorage to avoid re-prompting
  if (outcome === 'dismissed') {
    localStorage.setItem('install_prompt_dismissed', 'true');
    localStorage.setItem('install_prompt_dismissed_at', Date.now().toString()
  }
}

// Check if we should show the prompt
function shouldShowInstallPrompt() {
  const dismissed = localStorage.getItem('install_prompt_dismissed');
  if (!dismissed) return true;

  // Re-prompt after 30 days
  const dismissedAt = parseInt(localStorage.getItem('install_prompt_dismissed
  const daysSinceDismissed = (Date.now() - dismissedAt) / (1000 * 60 * 60 * 2
  return daysSinceDismissed > 30;
}
```

Exercise: Implement a custom install prompt for a PWA. Show it after the user has completed 3 interactions (e.g., clicked 3 buttons). Store the user's decision in localStorage and don't re-prompt for 7 days if they dismissed it.

CHAPTER 14

Updates & Enhancements

Service Worker Update Lifecycle

When you deploy a new Service Worker, it goes through a specific lifecycle:

1. Browser detects the new SW file (byte-different)
2. New SW installs in the background
3. New SW enters "waiting" state until all pages using the old SW are closed
4. New SW activates and takes control

```
// Detect Service Worker updates
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js').then(registration => {
    // Check for updates every minute
    setInterval(() => {
      registration.update();
    }, 60 * 1000);

    // Listen for new SW
    registration.addEventListener('updatefound', () => {
      const newWorker = registration.installing;
      newWorker.addEventListener('statechange', () => {
        if (newWorker.state === 'installed') {
          if (navigator.serviceWorker.controller) {
            // New version available!
            showUpdateNotification();
          }
        }
      });
    });
  });
}
```

Update Strategies

1. Immediate Update (Skip Waiting)

```
// sw.js
self.addEventListener('install', () => {
  self.skipWaiting();
});

self.addEventListener('activate', (event) => {
  event.waitUntil(self.clients.claim());
});

// page.js - Force update when user clicks "Update"
function applyUpdate() {
  if (window.newWorker) {
    window.newWorker.postMessage({ type: 'SKIP_WAITING' });
  }
}

// sw.js
self.addEventListener('message', (event) => {
  if (event.data.type === 'SKIP_WAITING') {
    self.skipWaiting();
  }
});
```

2. Gentle Update (Notify + Defer)

```
// Show a notification bar
function showUpdateNotification() {
  const banner = document.createElement('div');
  banner.className = 'update-banner';
  banner.innerHTML = `
    A new version is available.
    <button onclick="applyUpdate()">Update</button>
    <button onclick="dismissUpdate()">Later</button>
  `;
  document.body.prepend(banner);
}

// Listen for controller change (after update)
let refreshing = false;
navigator.serviceWorker.addEventListener('controllerchange', () => {
  if (refreshing) return;
  refreshing = true;
  window.location.reload();
});
```

Versioning

```
// sw.js - Use cache versioning
const CACHE_VERSION = 'v2';
const STATIC_CACHE = `static-${CACHE_VERSION}`;
const DYNAMIC_CACHE = `dynamic-${CACHE_VERSION}`;

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then(keys => {
      return Promise.all(
        keys.map(key => {
          if (key !== STATIC_CACHE && key !== DYNAMIC_CACHE) {
            return caches.delete(key);
          }
        })
      );
    })
  );
});
```

Progressive Enhancement

Progressive enhancement means your PWA should work as a basic website even without JavaScript, then enhance when more capabilities are available.

```
// Check feature availability
const features = {
  serviceWorker: 'serviceWorker' in navigator,
  cache: 'caches' in window,
  indexedDB: 'indexedDB' in window,
  notifications: 'Notification' in window,
  pushManager: 'PushManager' in window,
  sync: 'SyncManager' in window,
  bluetooth: 'bluetooth' in navigator,
  usb: 'usb' in navigator,
  nfc: 'nfc' in navigator
};

// Progressively enhance based on available features
if (features.serviceWorker) {
  // Register SW for offline support
}
if (features.notifications) {
  // Enable push notifications
}
if (features.sync) {
  // Enable background sync
}
```

> **Exercise:** Implement a versioned cache strategy for a PWA. When a new version is deployed, show a "New version available" toast notification. When the user clicks "Update", activate the new Service Worker and refresh the page.

CHAPTER 15

Progressive Enhancements

What is Progressive Enhancement?

Progressive enhancement is a strategy for web design that emphasizes core web content first, then layers on more advanced features for browsers that support them. PWAs are the epitome of progressive enhancement.

Feature Detection

Always check for feature support before using advanced APIs:

```
// Feature detection helper
function supports(feature) {
  switch (feature) {
    case 'service-worker':
      return 'serviceWorker' in navigator;
    case 'cache':
      return 'caches' in window;
    case 'indexeddb':
      return 'indexedDB' in window;
    case 'webgl':
      return !!window.WebGLRenderingContext;
    case 'webp':
      const elem = document.createElement('canvas');
      return elem.toDataURL('image/webp').indexOf('data:image/webp') === 0;
    case 'web-share':
      return navigator.share !== undefined;
    case 'clipboard':
      return navigator.clipboard !== undefined;
    case 'badging':
      return navigator.setAppBadge !== undefined;
    default:
      return false;
  }
}

// Usage
if (supports('web-share')) {
  shareButton.style.display = 'block';
}
```

Graceful Degradation

While progressive enhancement starts with the basics and adds features, graceful degradation starts with the full experience and scales back when features are missing.

```
// Graceful degradation example for push notifications
async function sendNotification(message) {
  if (!('Notification' in window)) {
    // Browser doesn't support notifications at all
    showInlineAlert(message);
    return;
  }

  if (Notification.permission === 'granted') {
    new Notification(message);
    return;
  }

  if (Notification.permission === 'default') {
    const permission = await Notification.requestPermission();
    if (permission === 'granted') {
      new Notification(message);
    } else {
      showInlineAlert(message);
    }
    return;
  }

  // Permission denied
  showInlineAlert(message);
}
```

Enhancement Layers

Think of PWA features in layers:

Layer	Features	Dependency
1 - Core	Semantic HTML, CSS, content	None
2 - Layout	Responsive design, Flexbox/Grid	Modern CSS
3 - Interactivity	JavaScript, animations	JS enabled
4 - Offline	Service Worker, Cache	HTTPS + modern browser
5 - App-like	Manifest, install prompt	Layer 4
6 - Advanced	Push, sync, badges, share	Layer 4 + permissions

Testing Enhancement Layers

Always test your PWA at each enhancement layer:

- Disable JavaScript — does the core content still render?
- Disable CSS — is the content still readable?
- Go offline — does the PWA still function?
- Use an older browser — do basic features work?
- Use a screen reader — is the app accessible?

> **Exercise:** Take a simple web page and progressively enhance it into a PWA. Start with just HTML (no CSS/JS), then add CSS for responsive layout, then JavaScript for interactivity, then a Service Worker for offline support, and finally a manifest for installation.

CHAPTER 16

Detection & Integration

Feature Detection

Detecting browser capabilities is essential for building resilient PWAs. Use feature detection rather than browser sniffing.

```
// Comprehensive feature detection
const pwaSupport = {
  serviceWorker: 'serviceWorker' in navigator,
  manifest: document.querySelector('link[rel="manifest"]') !== null,
  notifications: 'Notification' in window,
  push: 'PushManager' in window,
  sync: 'SyncManager' in window,
  cache: 'caches' in window,
  indexedDB: 'indexedDB' in window,
  webShare: navigator.share !== undefined,
  webShareTarget: 'shareTarget' in (document.querySelector('link[rel="manifes
  badging: navigator.setAppBadge !== undefined,
  credentials: 'credentials' in navigator,
  storage: navigator.storage !== undefined,
  connection: navigator.connection !== undefined,
  wakeLock: 'wakeLock' in navigator,
  screenWakeLock: 'wakeLock' in navigator,
  bluetooth: navigator.bluetooth !== undefined,
  usb: navigator.usb !== undefined,
  nfc: 'NDEFReader' in window,
  fileSystemAccess: 'showDirectoryPicker' in window,
  screenOrientation: screen.orientation !== undefined,
  clipboard: navigator.clipboard !== undefined,
  contacts: 'contacts' in navigator,
  geolocation: 'geolocation' in navigator,
  mediaDevices: navigator.mediaDevices !== undefined,
  internationalization: Intl !== undefined
};

// Use detection to show/hide features
if (pwaSupport.webShare) {
  document.getElementById('share-button').hidden = false;
}
```

Platform Detection

Detect the user's platform to provide platform-specific enhancements:

```
// Platform detection
const platform = {
  isAndroid: /android/i.test(navigator.userAgent),
  isIOS: /iphone|ipad|ipod/i.test(navigator.userAgent),
  isWindows: /windows nt/i.test(navigator.userAgent),
  isMacOS: /macintosh|mac os x/i.test(navigator.userAgent),
  isLinux: /linux/i.test(navigator.userAgent),
  isChromeOS: /cros/i.test(navigator.userAgent),
  isMobile: /mobile|android|iphone|ipad/i.test(navigator.userAgent),
  isStandalone: window.matchMedia('(display-mode: standalone)').matches
     || window.navigator.standalone === true,
  isInstalled: window.matchMedia('(display-mode: standalone)').matches
     || window.navigator.standalone === true
};

// Platform-specific adjustments
if (platform.isIOS) {
  // Add iOS-specific meta tags
  document.head.innerHTML += `
    <meta name="apple-mobile-web-app-capable" content="yes">
    <meta name="apple-mobile-web-app-status-bar-style" content="black-translu
  `;
}

if (platform.isStandalone) {
  // The app is running in standalone mode (installed)
  document.body.classList.add('is-installed');
}
```

Integration APIs

Web Share API

```
async function shareContent(title, text, url) {
  if (navigator.share) {
    try {
      await navigator.share({ title, text, url });
      console.log('Shared successfully');
    } catch (err) {
      console.error('Share failed:', err);
    }
  } else {
    // Fallback: copy to clipboard
    await navigator.clipboard.writeText(url);
    alert('Link copied to clipboard!');
  }
}
```

Credential Management

```
// Store credentials
async function storeCredentials(email, password) {
  if ('credentials' in navigator) {
    const cred = new PasswordCredential({
      id: email,
      password: password,
      name: 'My PWA'
    });
    await navigator.credentials.store(cred);
  }
}

// Auto-fill credentials
async function autoSignIn() {
  if ('credentials' in navigator) {
    const cred = await navigator.credentials.get({
      password: true,
      mediation: 'optional'
    });
    if (cred) {
      // Auto-fill the login form
      document.getElementById('email').value = cred.id;
      document.getElementById('password').value = cred.password;
    }
  }
}
```

Exercise: Create a feature detection dashboard that displays all supported PWA APIs on the user's browser. Use CSS grid to show each feature with a green checkmark (supported) or red X (not supported). Add the Web Share API as a share button on the page.

CHAPTER 17

OS Integration

Share Target API

Allow your PWA to receive shared content from other apps via the Share Target API.

```
// manifest.json - Register as a share target
{
  "share_target": {
    "action": "/share-handler",
    "method": "GET",
    "enctype": "application/x-www-form-urlencoded",
    "params": {
      "title": "title",
      "text": "text",
      "url": "url"
    }
  }
}

// HTML page for handling shared content
// share-handler.html
<!DOCTYPE html>
<html>
<head>
  <title>Share Handler</title>
</head>
<body>
  <script>
    const params = new URLSearchParams(window.location.search);
    const sharedTitle = params.get('title');
    const sharedText = params.get('text');
    const sharedUrl = params.get('url');

    if (sharedText) {
      // Process the shared content
      createNote(sharedText, sharedUrl);
      window.location.href = '/notes';
    }
  </script>
</body>
</html>
```

File Handling

Register your PWA to handle specific file types.

```
// manifest.json - File handling
{
  "file_handlers": [
    {
      "action": "/open-file",
      "accept": {
        "text/plain": [".txt", ".md"],
        "text/csv": [".csv"],
        "image/*": [".png", ".jpg", ".gif", ".webp"],
        "application/pdf": [".pdf"]
      }
    }
  ]
}

// Handle opened files
if ('launchQueue' in window) {
  window.launchQueue.setConsumer(async (launchParams) => {
    for (const file of launchParams.files) {
      const content = await file.getFile();
      const text = await content.text();
      // Process the file
      openDocument(text, content.name);
    }
  });
}
```

URL Handling

Register your PWA to handle URLs from your domain.

```
// manifest.json - URL handling
{
  "url_handlers": [
    {
      "origin": "https://my-pwa.example.com"
    }
  ]
}

// In the Service Worker, intercept navigation to shared URLs
self.addEventListener('fetch', (event) => {
  if (event.request.mode === 'navigate') {
    // Handle navigation to shared URLs
    const url = new URL(event.request.url);
    if (url.pathname.startsWith('/shared/')) {
      event.respondWith(handleSharedUrl(url));
    }
  }
});
```

Protocol Handlers

Register custom protocol handlers so your PWA opens when users click `web+myapp://` links.

```
// Register protocol handler
navigator.registerProtocolHandler('web+myapp', '/handle-protocol?url=%s');

// manifest.json
{
  "protocol_handlers": [
    {
      "protocol": "web+myapp",
      "url": "/handle-protocol?url=%s"
    }
  ]
}

// Handle protocol URL
// /handle-protocol.html
const url = new URLSearchParams(window.location.search).get('url');
if (url) {
  const parsed = new URL(url);
  const action = parsed.hostname;
  const data = parsed.pathname;
  // e.g., web+myapp://open-note/123
  if (action === 'open-note') {
    openNote(data.replace('/', ''));
  }
}
```

> **Exercise:** Create a PWA that registers as a share target for images. When a user shares an image to your PWA from another app, display it on the page and save it to IndexedDB. Also register the PWA to handle .txt files.

CHAPTER 18

Window Management

Display Modes

Use CSS media queries to detect and adapt to different display modes:

```
/* CSS display mode detection */
@media (display-mode: browser) {
  /* Regular browser tab */
  body::before { content: "Browser mode"; }
}

@media (display-mode: standalone) {
  /* Installed PWA */
  body { background: var(--app-bg); }
  .browser-chrome { display: none; }
}

@media (display-mode: minimal-ui) {
  /* Minimal browser UI */
  .minimal-ui-hint { display: block; }
}

@media (display-mode: fullscreen) {
  /* Full screen */
  body { cursor: none; }
}

/* JavaScript display mode detection */
const displayMode = window.matchMedia('(display-mode: standalone)').matches
  ? 'standalone'
  : window.matchMedia('(display-mode: minimal-ui)').matches
    ? 'minimal-ui'
    : window.matchMedia('(display-mode: fullscreen)').matches
      ? 'fullscreen'
      : 'browser';

console.log('Current display mode:', displayMode);
```

Window Controls Overlay

The Window Controls Overlay API lets you customize the title bar area of installed desktop PWAs.

```
// manifest.json - Enable overlay
{
  "display_override": ["window-controls-overlay"],
  "display": "standalone"
}

// CSS for the title bar area
#title-bar {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  height: 40px;
  display: flex;
  align-items: center;
  padding-left: env(titlebar-area-x, 0);
  padding-top: env(titlebar-area-y, 0);
  width: env(titlebar-area-width, 100%);
  height: env(titlebar-area-height, 40px);
  -webkit-app-region: drag;
  background: var(--theme-color);
}

#title-bar button {
  -webkit-app-region: no-drag;
}

// JavaScript API
if ('windowControlsOverlay' in navigator) {
  const overlay = navigator.windowControlsOverlay;

  // Check if overlay is visible
  console.log('Overlay visible:', overlay.visible);

  // Listen for geometry changes
  overlay.addEventListener('geometrychange', (event) => {
    const { x, y, width, height } = event.titlebarAreaRect;
    console.log('Title bar area:', { x, y, width, height });
  });
}
```

Multi-window Management

Open and manage multiple windows from your PWA:

```
// Open a new window
async function openDocumentWindow(docId) {
  const windowRef = window.open(
    `/document?id=${docId}`,
    'doc-window',
    'width=800,height=600'
  );

  // Communicate between windows
  windowRef.addEventListener('message', (event) => {
    if (event.data.type === 'DOCUMENT_SAVED') {
      refreshDocumentList();
    }
  });
}

// Post message to opener window
window.opener.postMessage({
  type: 'DOCUMENT_SAVED',
  docId: 123
}, '*');

// Get all windows (from Service Worker)
self.addEventListener('message', (event) => {
  self.clients.matchAll({ type: 'window' }).then(clients => {
    clients.forEach(client => {
      client.postMessage({ type: 'UPDATE_AVAILABLE' });
    });
  });
});
```

Screen API

```
// Keep screen on (Wake Lock)
async function keepScreenOn() {
  if ('wakeLock' in navigator) {
    try {
      const wakeLock = await navigator.wakeLock.request('screen');
      console.log('Wake lock acquired');

      wakeLock.addEventListener('release', () => {
        console.log('Wake lock released');
      });
    } catch (err) {
      console.error('Wake lock failed:', err);
    }
  }
}

// Detect screen orientation
screen.orientation.addEventListener('change', () => {
  console.log('Orientation:', screen.orientation.type);
});
```

Exercise: Create a PWA that uses the Window Controls Overlay for a custom title bar with the app name and a "minimize to tray" action. Also implement a multi-window note editor where each note opens in a separate window that can post messages back to the main window.

CHAPTER 19

Experimental Features

This chapter covers cutting-edge web capabilities that are still in development or available behind experimental flags. These APIs may change or have limited browser support.

Origin Trials

Origin Trials allow developers to test experimental APIs with real users before they become standards. Register for a token at developer.chrome.com/origintrials.

```
<!-- Add the origin trial meta tag -->
<meta http-equiv="origin-trial" content="YOUR_TOKEN_HERE">

// Or via HTTP header
// Origin-Trial: YOUR_TOKEN_HERE
```

Future Capabilities

File System Access API

```
// Read a file from the local file system
async function openFile() {
  try {
    const [handle] = await window.showOpenFilePicker({
      types: [{
        description: 'Text Files',
        accept: { 'text/plain': ['.txt', '.md'] }
      }]
    });
    const file = await handle.getFile();
    const content = await file.text();
    return { handle, content };
  } catch (err) {
    console.error('File open cancelled:', err);
  }
}

// Write back to the file
async function saveFile(handle, content) {
  const writable = await handle.createWritable();
  await writable.write(content);
  await writable.close();
}

// Create a new file
async function createFile(name) {
  const handle = await window.showSaveFilePicker({
    suggestedName: name,
    types: [{
      description: 'Text Files',
      accept: { 'text/plain': ['.txt', '.md'] }
    }]
  });
  return handle;
}
```

Local Font Access API

```
// Access locally installed fonts
async function getLocalFonts() {
  if ('queryLocalFonts' in window) {
    const fonts = await window.queryLocalFonts();
    return fonts.map(font => ({
      family: font.family,
      style: font.style,
      fullName: font.fullName
    }));
  }
  return [];
}
```

Screen Details API

```
// Get detailed screen information
async function getScreenDetails() {
  if ('getScreenDetails' in window) {
    const screens = await window.getScreenDetails();
    console.log('Number of screens:', screens.screens.length);

    screens.screens.forEach(screen => {
      console.log('Screen:', {
        id: screen.id,
        width: screen.width,
        height: screen.height,
        availWidth: screen.availWidth,
        availHeight: screen.availHeight,
        isPrimary: screen.isPrimary,
        isInternal: screen.isInternal
      });
    });

    // Move a window to a specific screen
    window.moveTo(screens.screens[1].availLeft, 0);
  }
}
```

Compute Pressure API

```
// Monitor system pressure (CPU/thermal)
async function monitorPressure() {
  if ('computePressure' in navigator) {
    const observer = new ComputePressureObserver((update) => {
      console.log('CPU Pressure:', update.cpuUtilization);
      console.log('Thermal State:', update.thermalState);

      if (update.cpuUtilization > 0.8) {
        // Reduce app complexity
        reduceAnimationQuality();
      }
    }, { cpuUtilization: true, thermalState: true });

    await observer.observe();
  }
}
```

Declarative Link Capturing (DLC)

```
// manifest.json - Handle links within the PWA scope
{
  "capture_links": "existing-client-event",
  "launch_handler": {
    "client_mode": ["focus-existing", "navigate-existing"]
  }
}
```

> **Note:** Experimental APIs may change, break, or be removed. Always use feature detection and provide fallbacks. Check chromestatus.com for the latest status.

Exercise: Enable an origin trial for the File System Access API and build a simple text editor that can open, edit, and save local files. Show a warning if the API is not available and provide a localStorage-based fallback.

CHAPTER 20

Tools & Debugging

Chrome DevTools for PWAs

Chrome DevTools provides dedicated panels for debugging PWAs:

Application Panel

- **Manifest:** Validate and preview the Web App Manifest
- **Service Workers:** Inspect SW status, push to debug, unregister, update cycle
- **Cache Storage:** View and delete cache entries
- **IndexedDB:** Browse, edit, and delete IndexedDB data
- **Storage:** View storage usage by origin
- **Background Services:** Inspect background sync, push messaging, notifications

Debugging Service Workers

```
// Add logging in Service Worker for easier debugging
const DEBUG = true;

function log(...args) {
  if (DEBUG) {
    console.log('[SW]', ...args);
  }
}

self.addEventListener('install', (event) => {
  log('Installing...');
  // ...
});

self.addEventListener('fetch', (event) => {
  log('Fetch:', event.request.url);
  // ...
});

// To see SW logs:
// 1. Open DevTools > Application > Service Workers
// 2. Click "Inspect" to open the SW console
// 3. All console.log calls from the SW appear here
```

Testing PWAs

Lighthouse Audits

```
# Run Lighthouse from CLI
npm install -g lighthouse
lighthouse https://my-pwa.com --view

# Or use in CI
lighthouse https://my-pwa.com \
  --chrome-flags="--headless" \
  --output=json \
  --output-path=./report.json \
  --preset=desktop
```

Automated Testing

```
// Using Puppeteer for PWA testing
const puppeteer = require('puppeteer');

(async () => {
  const browser = await puppeteer.launch();
  const page = await browser.newPage();

  // Enable DevTools Protocol for SW inspection
  const cdp = await page.target().createCDPSession();
  await cdp.send('ServiceWorker.enable');

  // Listen for SW events
  cdp.on('ServiceWorker.workerRegistrationUpdated', (event) => {
    console.log('Registrations:', event.registrations);
  });

  await page.goto('https://my-pwa.com');

  // Check if SW is active
  const hasSW = await page.evaluate(() => {
    return 'serviceWorker' in navigator;
  });
  console.log('Has Service Worker:', hasSW);

  // Test offline behavior
  await page.setOfflineMode(true);
  await page.reload({ waitUntil: 'networkidle0' });
  const bodyText = await page.evaluate(() => document.body.innerText);
  console.log('Offline content:', bodyText);

  await browser.close();
})();
```

PWA Checklist

Check	How to Verify
HTTPS	Check URL bar for lock icon
Manifest valid	DevTools > Application > Manifest
SW registered	DevTools > Application > Service Workers
Offline page	Go offline in DevTools, reload
Fast load	Lighthouse performance audit
Install prompt	Trigger beforeinstallprompt
Responsive	Device toolbar in DevTools
Accessible	Lighthouse accessibility audit

> **Exercise:** Run a Lighthouse audit on your PWA and aim for a score of at least 90 in all categories (Performance, Accessibility, Best Practices, SEO, PWA). Fix any failing audits and re-run until you pass.

CHAPTER 21

Architecture & Design Patterns

App Shell Pattern

The App Shell pattern separates the minimal HTML, CSS, and JavaScript needed to render the user interface from the dynamic content. This shell is cached on first load for instant rendering on subsequent visits.

```
// App Shell Architecture
// 1. Cache the shell on install
self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open('shell-v1').then(cache => {
      return cache.addAll([
        '/shell/',
        '/shell/styles.css',
        '/shell/app.js',
        '/shell/header.html',
        '/shell/footer.html',
        '/shell/nav.html',
        '/offline.html'
      ]);
    })
  );
});

// 2. Serve shell from cache, content from network
self.addEventListener('fetch', (event) => {
  const url = new URL(event.request.url);

  // Shell resources: cache first
  if (url.pathname.startsWith('/shell/')) {
    event.respondWith(caches.match(event.request));
    return;
  }

  // Content pages: network first with cache fallback
  if (url.pathname.startsWith('/content/')) {
    event.respondWith(
      fetch(event.request)
        .catch(() => caches.match('/offline.html'))
    );
    return;
  }

  // API calls: network only
  if (url.pathname.startsWith('/api/')) {
    event.respondWith(fetch(event.request));
    return;
  }
});
```

PRPL Pattern

PRPL is a pattern for structuring and serving PWAs for fast initial load:

- **P**ush critical resources for the initial URL route.
- **R**ender the initial route as soon as possible.
- **P**re-cache remaining routes and assets.
- **L**azy-load routes on demand.

RAIL Model

RAIL is a user-centric performance model:

Letter	Meaning	Target
R	Response	50ms for input response
A	Animation	10ms per frame (60fps)
I	Idle	Use idle time for deferred work
L	Load	1s for content-first paint

Scalability Considerations

- **Lazy loading:** Load routes and components on demand
- **Code splitting:** Split JS bundles by route or component
- **Dynamic imports:** `import('./module.js')` for on-demand loading
- **Intersection Observer:** Lazy load images and components when they enter the viewport
- **Virtual scrolling:** For large lists, only render visible items

```
// Dynamic imports for code splitting
const button = document.getElementById('load-chart');
button.addEventListener('click', async () => {
  const { renderChart } = await import('./charts.js');
  renderChart('revenue-data');
});

// Intersection Observer for lazy loading
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      observer.unobserve(img);
    }
  });
});

document.querySelectorAll('img[data-src]').forEach(img => {
  observer.observe(img);
});
```

Exercise: Refactor a simple SPA to use the App Shell pattern. Cache the shell (header, footer, navigation) on install, serve it from cache, and lazy-load the content pages on demand. Measure the performance improvement using Lighthouse.

CHAPTER 22

Complexity Management

State Management

Managing application state in a PWA requires handling both online and offline states, sync status, and multiple clients.

```
// Centralized state management for PWAs
const PWAState = {
  _state: {
    online: navigator.onLine,
    syncNeeded: false,
    pendingActions: [],
    lastSync: null,
    user: null,
    settings: {}
  },

  _listeners: [],

  get(key) {
    return this._state[key];
  },

  set(key, value) {
    this._state[key] = value;
    this._notify(key, value);
  },

  subscribe(listener) {
    this._listeners.push(listener);
    return () => {
      this._listeners = this._listeners.filter(l => l !== listener);
    };
  },

  _notify(key, value) {
    this._listeners.forEach(listener => {
      try {
        listener(key, value, this._state);
      } catch (err) {
        console.error('State listener error:', err);
      }
    });
  },

  // Persist state to localStorage
  persist() {
    localStorage.setItem('pwa_state', JSON.stringify(this._state));
  },
```

```
  // Restore state from localStorage
  restore() {
    try {
      const saved = localStorage.getItem('pwa_state');
      if (saved) {
        this._state = { ...this._state, ...JSON.parse(saved) };
      }
    } catch (err) {
      console.error('Failed to restore state:', err);
    }
  }
};

// Online/Offline tracking
window.addEventListener('online', () => {
  PWAState.set('online', true);
  syncPendingActions();
});

window.addEventListener('offline', () => {
  PWAState.set('online', false);
});
```

Code Organization

```
// Recommended PWA project structure
my-pwa/
  src/
    app/
      shell.js          # App shell logic
      router.js         # Client-side routing
      state.js          # State management
    sw/
      sw.js             # Service Worker entry
      caching.js        # Caching strategies
      sync.js           # Background sync
      push.js           # Push notifications
    pages/
      home/
        index.html
        home.js
        home.css
      settings/
        index.html
        settings.js
        settings.css
    components/
      header.js
      footer.js
      offline-banner.js
    utils/
      db.js             # IndexedDB helpers
      network.js        # Network detection
      features.js       # Feature detection
    assets/
      images/
      fonts/
      icons/
  manifest.json
  index.html
```

Bundling and Build Tools

```
// Vite config for PWA
import { defineConfig } from 'vite';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['fonts/*.woff2', 'images/*.{png,jpg,webp}'],
      manifest: {
        name: 'My PWA',
        short_name: 'MyPWA',
        display: 'standalone',
        theme_color: '#4f46e5',
        icons: [
          { src: '/icons/icon-192.png', sizes: '192x192', type: 'image/png' }
          { src: '/icons/icon-512.png', sizes: '512x512', type: 'image/png' }
        ]
      },
      workbox: {
        globPatterns: ['**/*.{js,css,html,ico,png,jpg,webp,woff2}'],
        runtimeCaching: [
          {
            urlPattern: /\/api\//,
            handler: 'NetworkFirst',
            options: {
              cacheName: 'api-cache',
              expiration: { maxEntries: 50, maxAgeSeconds: 300 }
            }
          }
        ]
      }
    })
  ]
});
```

Complexity Patterns

- **Module Pattern:** Encapsulate related functionality in modules

- **Observer Pattern:** Event-driven communication between components
- **Mediator Pattern:** Centralized communication hub
- **Repository Pattern:** Abstract data storage behind a consistent API
- **Strategy Pattern:** Swap algorithms (e.g., different caching strategies)

> **Exercise:** Refactor a PWA with messy state management into a clean architecture using the state management pattern above. Add online/offline detection, persist state to localStorage, and ensure the UI reactively updates when the state changes.

CHAPTER 23

Capabilities

Push Notifications

Push notifications allow your PWA to send messages to users even when the browser is not open. They require two steps: subscribing to push notifications and displaying them.

Subscribing to Push

```
// Convert VAPID public key to Uint8Array
function urlBase64ToUint8Array(base64String) {
  const padding = '='.repeat((4 - base64String.length % 4) % 4);
  const base64 = (base64String + padding)
    .replace(/-/g, '+')
    .replace(/_/g, '/');
  const rawData = window.atob(base64);
  const output = new Uint8Array(rawData.length);
  for (let i = 0; i < rawData.length; ++i) {
    output[i] = rawData.charCodeAt(i);
  }
  return output;
}

// Subscribe to push
async function subscribeToPush() {
  const registration = await navigator.serviceWorker.ready;
  const subscription = await registration.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: urlBase64ToUint8Array(
      'YOUR_VAPID_PUBLIC_KEY'
    )
  });

  // Send subscription to your server
  await fetch('/api/push/subscribe', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(subscription)
  });

  return subscription;
}

// Check and request permission
async function enableNotifications() {
  if (!('Notification' in window)) {
    console.log('Notifications not supported');
    return false;
  }

  const permission = await Notification.requestPermission();
```

```
  return permission === 'granted';
}
```

Service Worker - Display Notification

```
// sw.js - Handle push events
self.addEventListener('push', (event) => {
  let data = { title: 'New Message', body: 'You have a new notification', ico

  if (event.data) {
    try {
      data = event.data.json();
    } catch {
      data.body = event.data.text();
    }
  }

  event.waitUntil(
    self.registration.showNotification(data.title, {
      body: data.body,
      icon: data.icon,
      badge: '/badge-72.png',
      vibrate: [200, 100, 200],
      data: data,
      actions: [
        { action: 'open', title: 'Open' },
        { action: 'dismiss', title: 'Dismiss' }
      ]
    })
  );
});

// Handle notification clicks
self.addEventListener('notificationclick', (event) => {
  event.notification.close();

  if (event.action === 'dismiss') return;

  const urlToOpen = event.notification.data?.url || '/';

  event.waitUntil(
    self.clients.matchAll({ type: 'window' }).then(clients => {
      for (const client of clients) {
        if (client.url === urlToOpen && 'focus' in client) {
          return client.focus();
        }
      }
```

```
      return self.clients.openWindow(urlToOpen);
    })
  );
});
```

Background Sync

```
// Register sync for pending data
async function queueAction(actionData) {
  await saveToIndexedDB('pending-actions', actionData);

  const registration = await navigator.serviceWorker.ready;
  await registration.sync.register('sync-actions');
}

// In Service Worker
self.addEventListener('sync', (event) => {
  if (event.tag === 'sync-actions') {
    event.waitUntil(processPendingActions());
  }
});

async function processPendingActions() {
  const actions = await getAllFromIndexedDB('pending-actions');
  for (const action of actions) {
    try {
      await fetch(action.url, {
        method: action.method,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(action.data)
      });
      await deleteFromIndexedDB('pending-actions', action.id);
    } catch (err) {
      console.error('Sync failed for action', action.id, err);
    }
  }
}
```

Advanced APIs

Badging API

```
// Set app badge
async function setBadge(count) {
  if (navigator.setAppBadge) {
    await navigator.setAppBadge(count);
  } else if (navigator.setExperimentalAppBadge) {
    await navigator.setExperimentalAppBadge(count);
  }
}

// Clear badge
async function clearBadge() {
  if (navigator.clearAppBadge) {
    await navigator.clearAppBadge();
  }
}

// Update badge when unread count changes
function updateBadge(unreadCount) {
  if (unreadCount > 0) {
    setBadge(unreadCount);
  } else {
    clearBadge();
  }
}
```

Media Capabilities

```
// Check media decoding capabilities
async function checkMediaSupport() {
  const config = {
    type: 'file',
    video: {
      contentType: 'video/webm; codecs="vp9"',
      width: 1920,
      height: 1080,
      bitrate: 5000000,
      framerate: 30
    }
  };

  const support = await navigator.mediaCapabilities.decodingInfo(config);
  console.log('Supported:', support.supported);
  console.log('Smooth:', support.smooth);
  console.log('Power efficient:', support.powerEfficient);
}
```

Payment Request API

```
async function processPayment(items) {
  if (!window.PaymentRequest) {
    showFallbackPayment();
    return;
  }

  const methodData = [
    { supportedMethods: 'basic-card' },
    { supportedMethods: 'https://google.com/pay' }
  ];

  const details = {
    total: {
      label: 'Total',
      amount: { currency: 'USD', value: items.reduce((sum, i) => sum + i.pric
    },
    displayItems: items.map(item => ({
      label: item.name,
      amount: { currency: 'USD', value: item.price.toFixed(2) }
    }))
  };

  try {
    const request = new PaymentRequest(methodData, details);
    const response = await request.show();
    await response.complete('success');
    return response.details;
  } catch (err) {
    console.error('Payment failed:', err);
    return null;
  }
}
```

Exercise: Build a PWA that sends push notifications when new content is available. Implement a "subscribe" button that requests notification permission, subscribes to push (mock the server side), and displays a notification when triggered. Also add a badge showing the count of unread items.

CHAPTER 24

Conclusion & Resources

Summary

Congratulations! You've completed the Progressive Web Apps Tutorial. Let's recap what you've learned:

- **What PWAs are** and why they matter
- **Service Workers** — lifecycle, registration, event handling
- **Caching strategies** — offline-first, network-first, stale-while-revalidate
- **Web App Manifest** — installation, display modes, icons
- **Workbox** — production-ready SW library
- **Offline data** — IndexedDB, Background Sync, Cache API
- **Install prompts** — customizing the install experience
- **OS integration** — Share Target, File Handling, Protocol Handlers
- **Push notifications** — engaging users even when the browser is closed
- **Performance** — App Shell, PRPL, RAIL model
- **Architecture** — scaling, complexity management, patterns
- **Experimental features** — cutting-edge APIs and origin trials

> **You now have the skills to build production-quality PWAs that work offline, load instantly, and feel like native apps.**

Quick Reference Guide

Feature	Key Resource
Service Workers	MDN: Service Worker API
Caching	Google DevTools > Application > Cache Storage
Manifest	Web App Manifest specification (W3C)
Workbox	developer.chrome.com/docs/workbox
IndexedDB	MDN: IndexedDB API / Dexie.js
Push	MDN: Push API / Web Push Protocol
Testing	Lighthouse, Puppeteer, DevTools
Best Practices	web.dev/progressive-web-apps

Cheatsheet

```
// Essential PWA Registration
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('/sw.js');
  });
}

// Essential Service Worker
self.addEventListener('install', e => e.waitUntil(
  caches.open('v1').then(c => c.addAll(['/']))
));
self.addEventListener('fetch', e => e.respondWith(
  caches.match(e.request) || fetch(e.request)
));

// Essential Manifest
{
  "name": "My PWA",
  "short_name": "PWA",
  "display": "standalone",
  "start_url": "/",
  "icons": [{ "src": "/icon-192.png", "sizes": "192x192" }]
}
```

Study Plan

Week	Topics	Goal
1	Ch1-5	Understand PWA fundamentals and design
2	Ch6-10	Master Service Workers, caching, and offline
3	Ch11-15	Installation, manifest, enhancement
4	Ch16-20	Integration, debugging, architecture
5	Ch21-25	Advanced capabilities, references

Certification Path

- Build and deploy 3 PWAs (simple, intermediate, advanced)
- Pass Google's PWA assessment on web.dev
- Contribute to an open-source PWA project
- Consider Google Mobile Web Specialist certification

> **Final Project:** Build a complete "Offline-first Notes" PWA with the following features: Create/read/update/delete notes, store in IndexedDB, sync to a server when back online, push notifications for reminders, installable with a custom install prompt, and a custom title bar using Window Controls Overlay. Score 90+ on Lighthouse.

CHAPTER 25

References

APIs & Specifications

API / Spec	Description	Status
Service Workers	Background script for offline, caching, push	W3C Candidate Recommendation
Web App Manifest	App metadata for installation	W3C Recommendation
Cache API	Request/Response pair storage	W3C Candidate Recommendation
IndexedDB	Client-side NoSQL database	W3C Recommendation
Push API	Server push to service workers	W3C Candidate Recommendation
Notification API	System notifications	W3C Recommendation
Background Sync	Deferred sync on connectivity	W3C Working Draft
Web Share API	Share content to other apps	W3C Candidate Recommendation
Share Target API	Receive shared content	W3C Draft Community Group
Badging API	App icon badges	W3C Candidate Recommendation
File System Access	Read/write local files	W3C Draft
Window Controls Overlay	Custom title bar in installed PWAs	W3C Draft

Essential Tools

- **Chrome DevTools:** Application panel for PWA debugging
- **Lighthouse:** Automated PWA auditing
- **Workbox:** Service Worker library by Google
- **PWA Builder:** PWA packaging for app stores
- **Webhint:** Web best practices linting
- **Puppeteer:** Browser automation for testing
- **Vite PWA Plugin:** Zero-config PWA setup
- **PWABuilder.com:** PWA analysis and packaging
- **Maskable.app:** Create maskable icons
- **Realfavicongenerator:** Generate all platform icons

Useful Resources

- **web.dev/progressive-web-apps:** Google's official PWA learning path
- **developer.mozilla.org - PWA:** MDN's comprehensive PWA documentation
- **whatpwacando.today:** Live demos of PWA capabilities
- **pwastats.com:** Browser support tracking for PWA features
- **web.dev/learn/pwa:** Google's structured PWA course
- **w3c.github.io/manifest/:** W3C Web App Manifest specification
- **w3c.org/TR/service-workers:** W3C Service Worker specification
- **chromestatus.com:** Chrome platform status for web features
- **caniuse.com:** Browser compatibility tables
- **github.com/GoogleChromeLabs:** Google Chrome Labs on GitHub

Community

- **Stack Overflow:** #progressive-web-apps tag
- **Chromium Blog:** Latest PWA news from Chrome team
- **Web.dev Discord:** Community discussions
- **Flig-PWA:** Spanish PWA community
- **Open Web Docs:** Web documentation community

Final Note

> **The web is the most powerful platform we have.** PWAs are the future of the web — they combine the reach of the web with the capabilities of native apps. Keep building, experiment with new APIs, and contribute to the web platform. The best is yet to come.